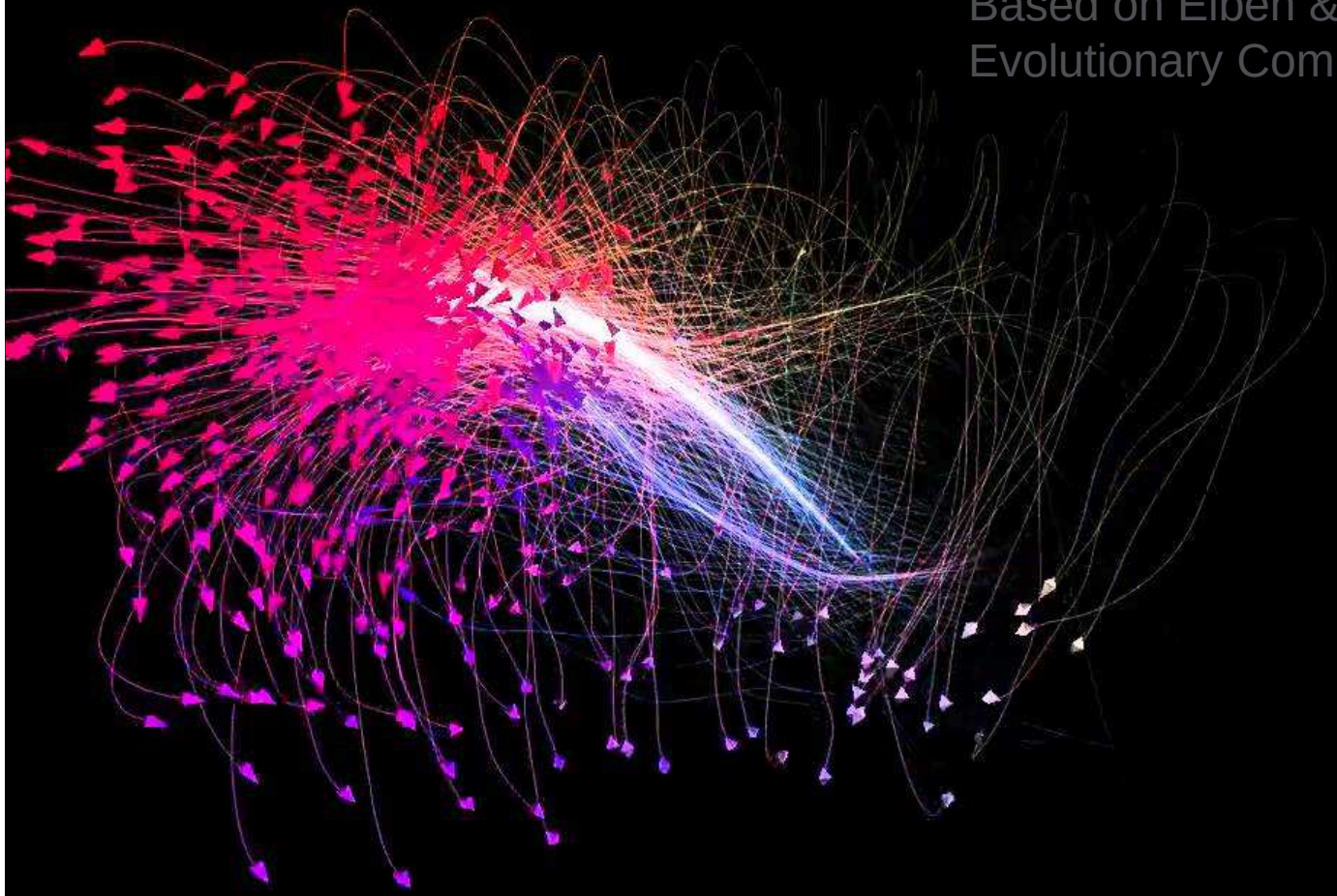
CS4720: Computational Intelligence

Genetic programming

Based on Eiben & Smith: Introduction to Evolutionary Computing, Chapters 4, 6



Prof Anikó Ekárt Dr Alina Patelli

Aston University

Outline

By the end of this lecture you should understand

- The genetic programming paradigm
- The differences between genetic programming and genetic algorithms
- Types of problems that can be solved by genetic programming



Outline cont’d

By the end of this lecture you should understand

- How GP works step by step
- A few “tricks of the trade”
- In what problems GP is expected to do well
- The Human-Competitive Results competition



THE GP PARADIGM

Historical perspective

1948, Turing:

proposes “genetical or evolutionary search”

1962, Bremermann:

optimisation through evolution and recombination

1964, Rechenberg:

introduces evolution strategies

1965, Fogel, Owens and Walsh:

introduce evolutionary programming

1975, Holland:

introduces genetic algorithms

1985, Cramer: introduces tree-based genetic programming

1992, Koza: expands genetic programming

1996, Keller & Banzhaf: Linear GP

1999, Miller: Cartesian GP

1999, O’Neill: Grammatical evolution



The inspiration for genetic programming

- Darwinian evolution: survival of the fittest
- Automatic programming:
  would it not be nice if we could instruct the computer what we wanted a program for and it created the program for us?
- Genetic programming ~to some extent~ is the answer
- If we specify desired (input, output) pairs and the possible elements, GP will produce programs consisting of some combination of those elements. The best program will match as closely as possible the (input, output) pairs.

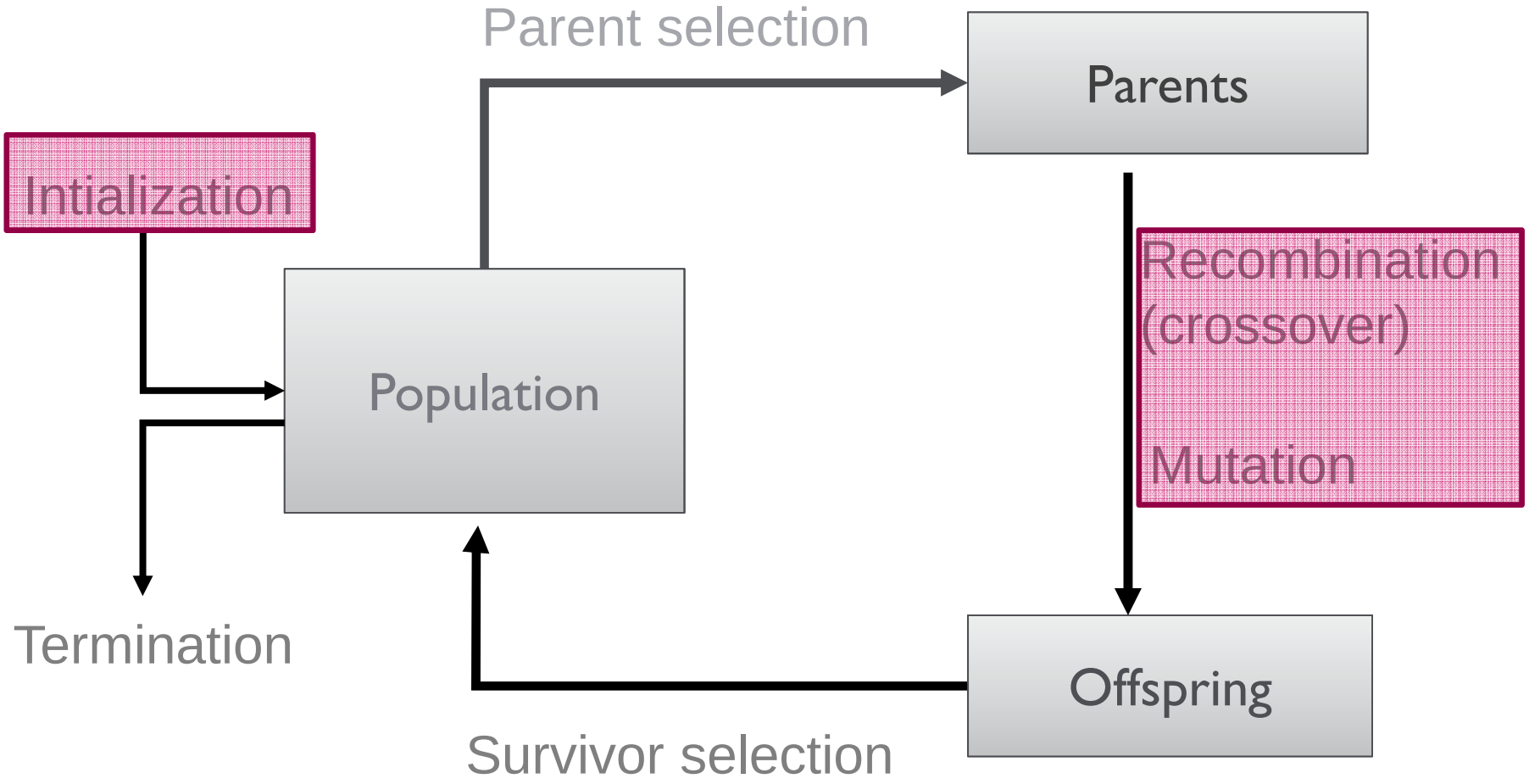


Example problem: symbolic regression

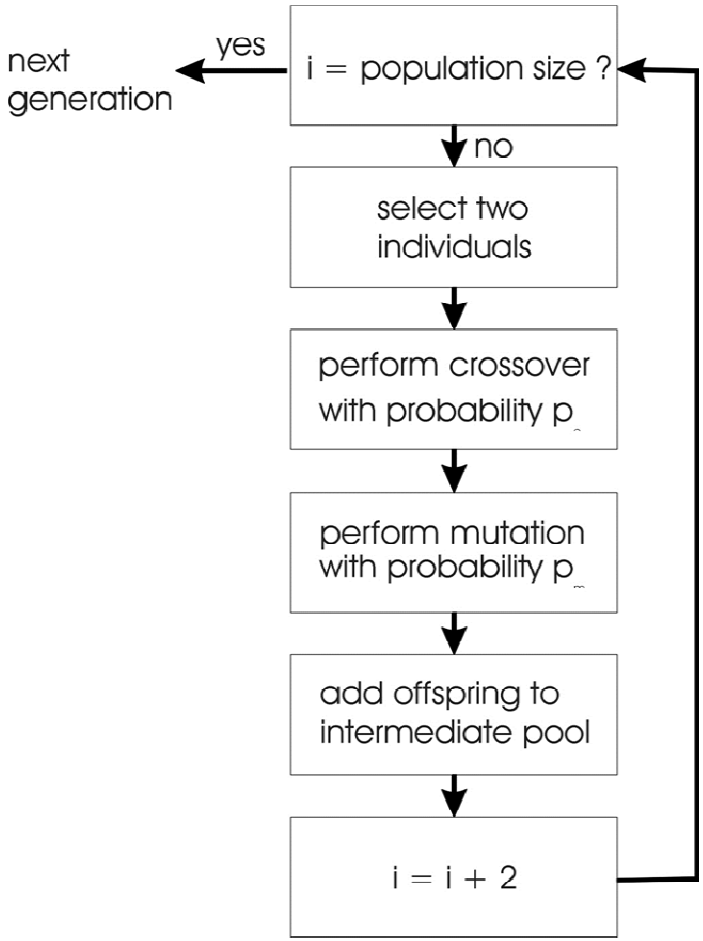
- Given a set of pairs of numbers (x_i, y_i), find the function $f(x)$ so that $f(x_i) = y_i$
- The function can include any mathematical functions and numerical constants



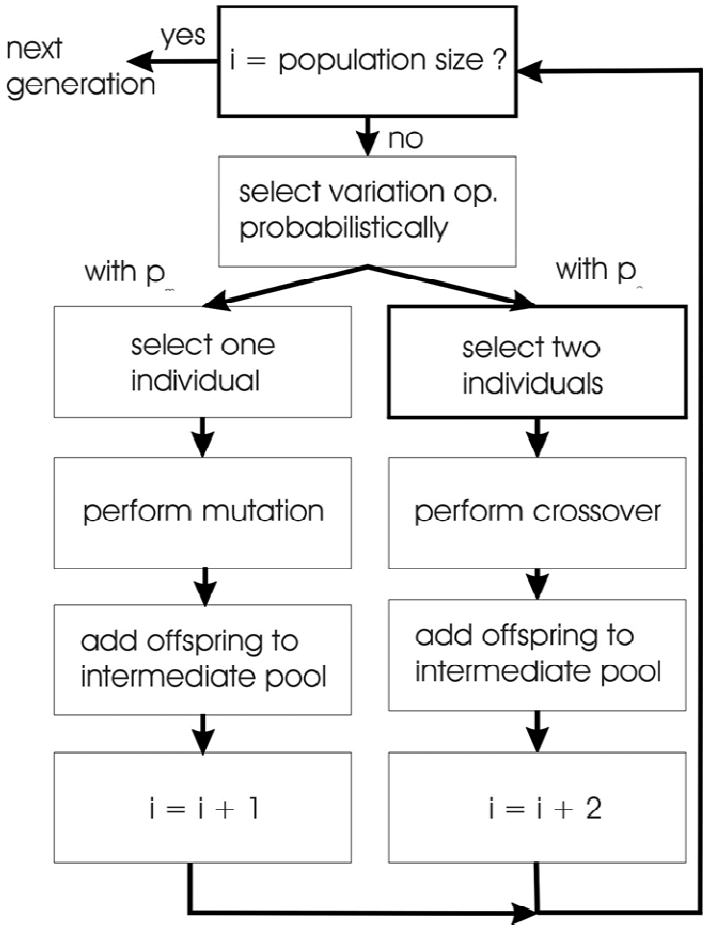
General working of evolutionary algorithms



Genetic programming vs genetic algorithm



GA flowchart



GP flowchart

Main components of GP vs GAs

- Representation: different
- Fitness evaluation: different

- Variation operators: different
- Initialisation: different



REPRESENTATION



Representation

- Generally
  - phenotype: object in original problem context
  - genotype: code to denote that object (chromosome)

- In GP this is more blurred:

  program ≈ program

(In grammatical evolution there is clear distinction)



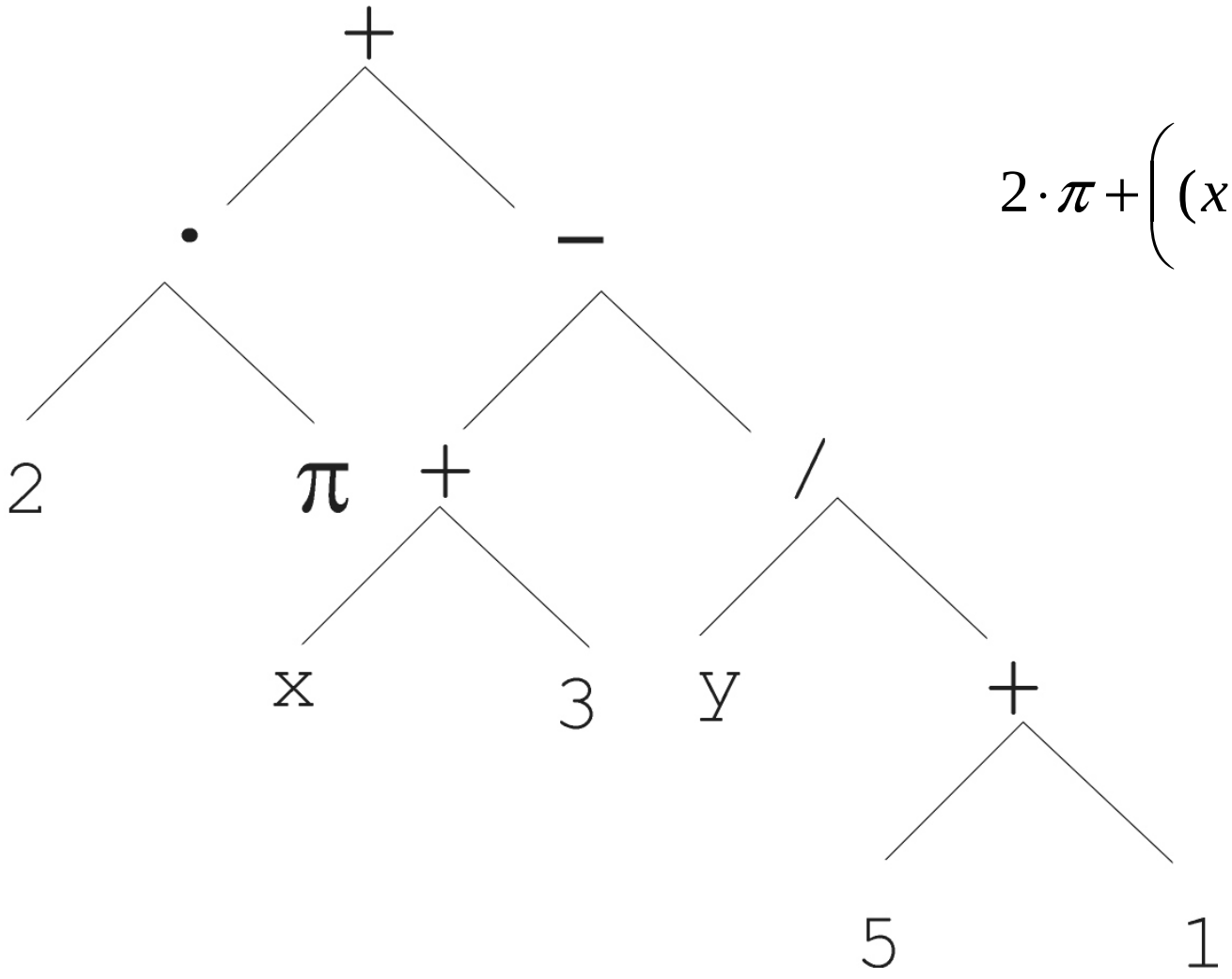
Tree Representation is universal

- Arithmetic formula: $2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$
- Logic formula: $(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$
- Program:

```
i =1;
while (i < 20)
{
        i = i +1;
}
```

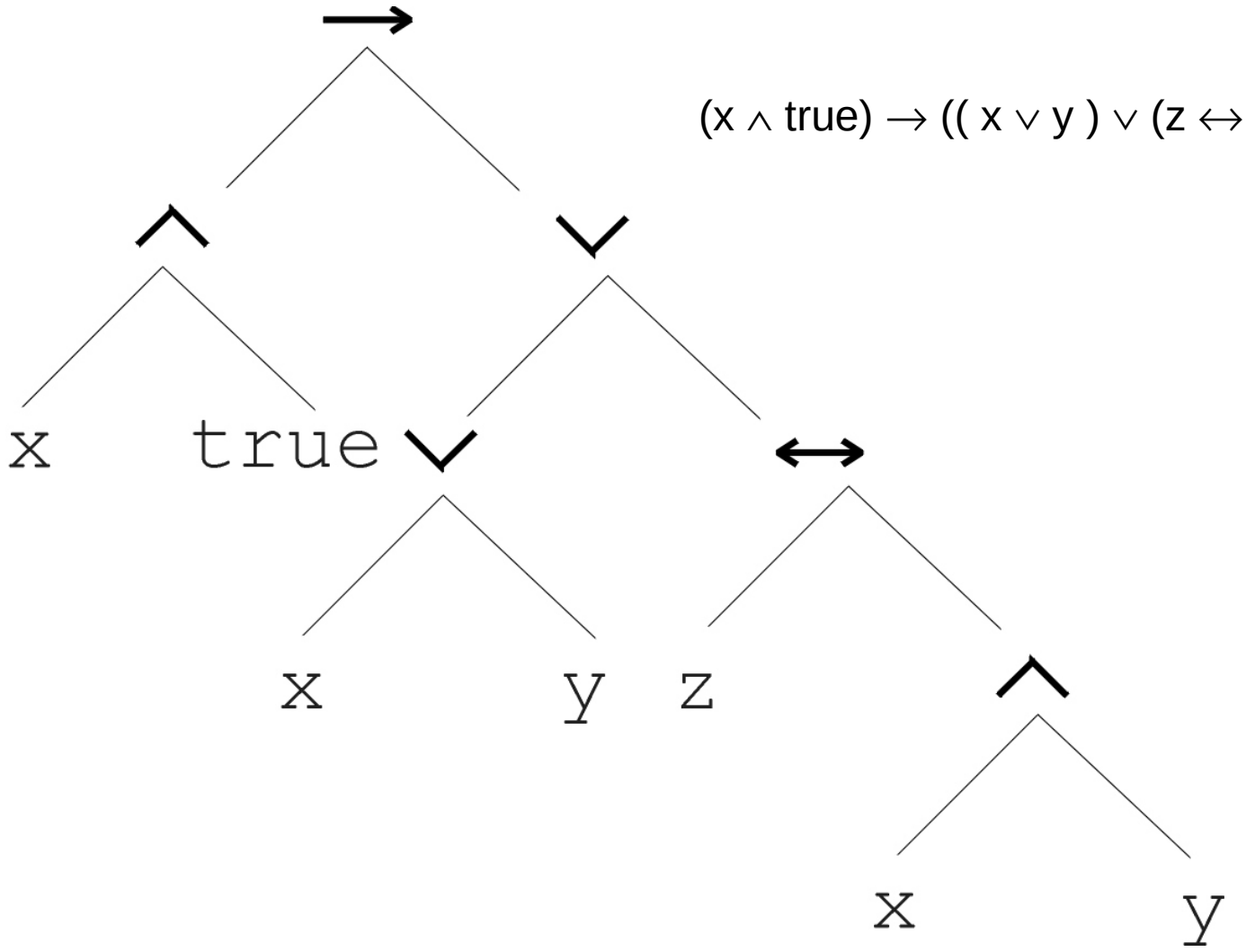


Arithmetic formula



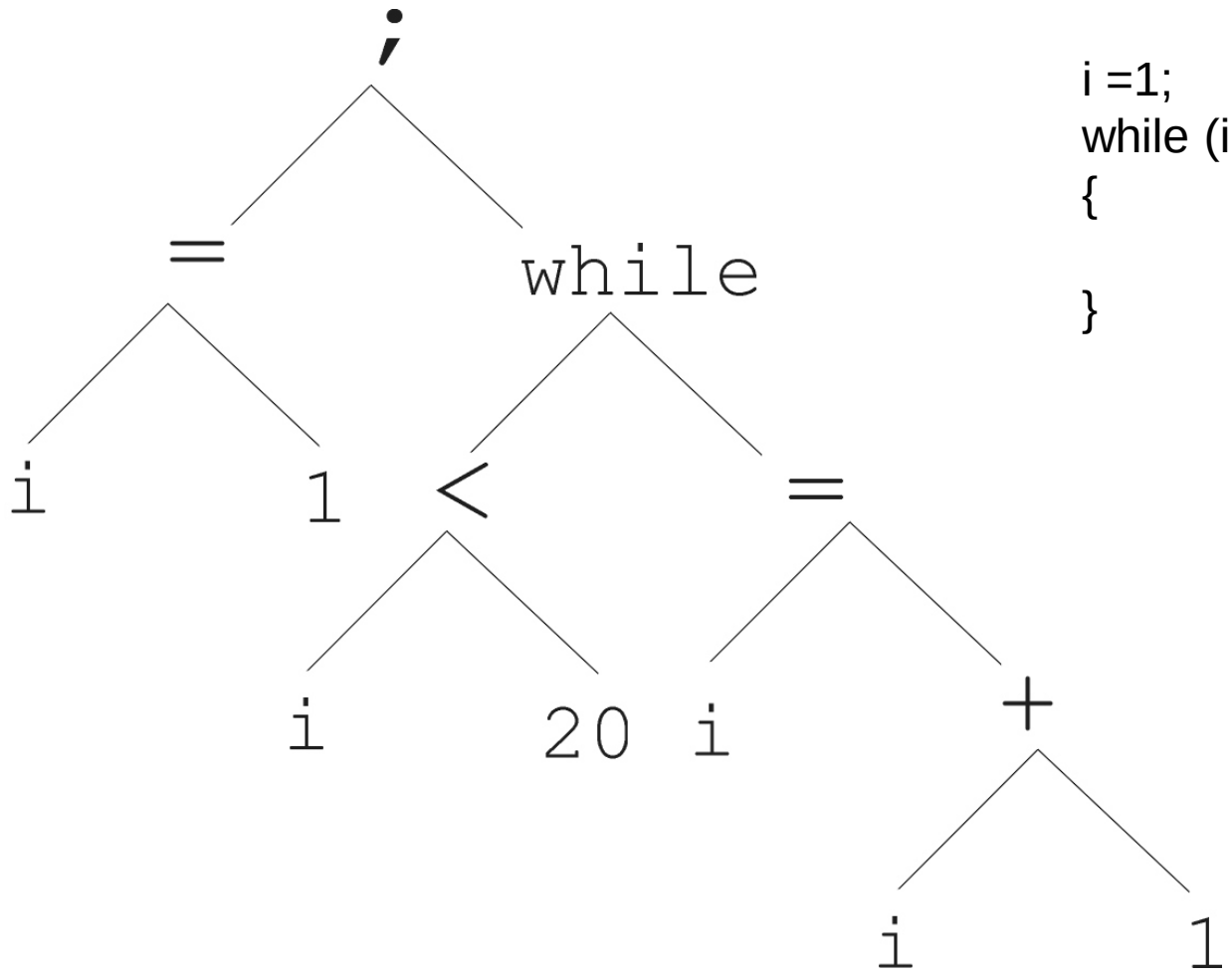
$$2 \cdot \pi + \left((x+3) - \frac{y}{5+1} \right)$$

Logic expression



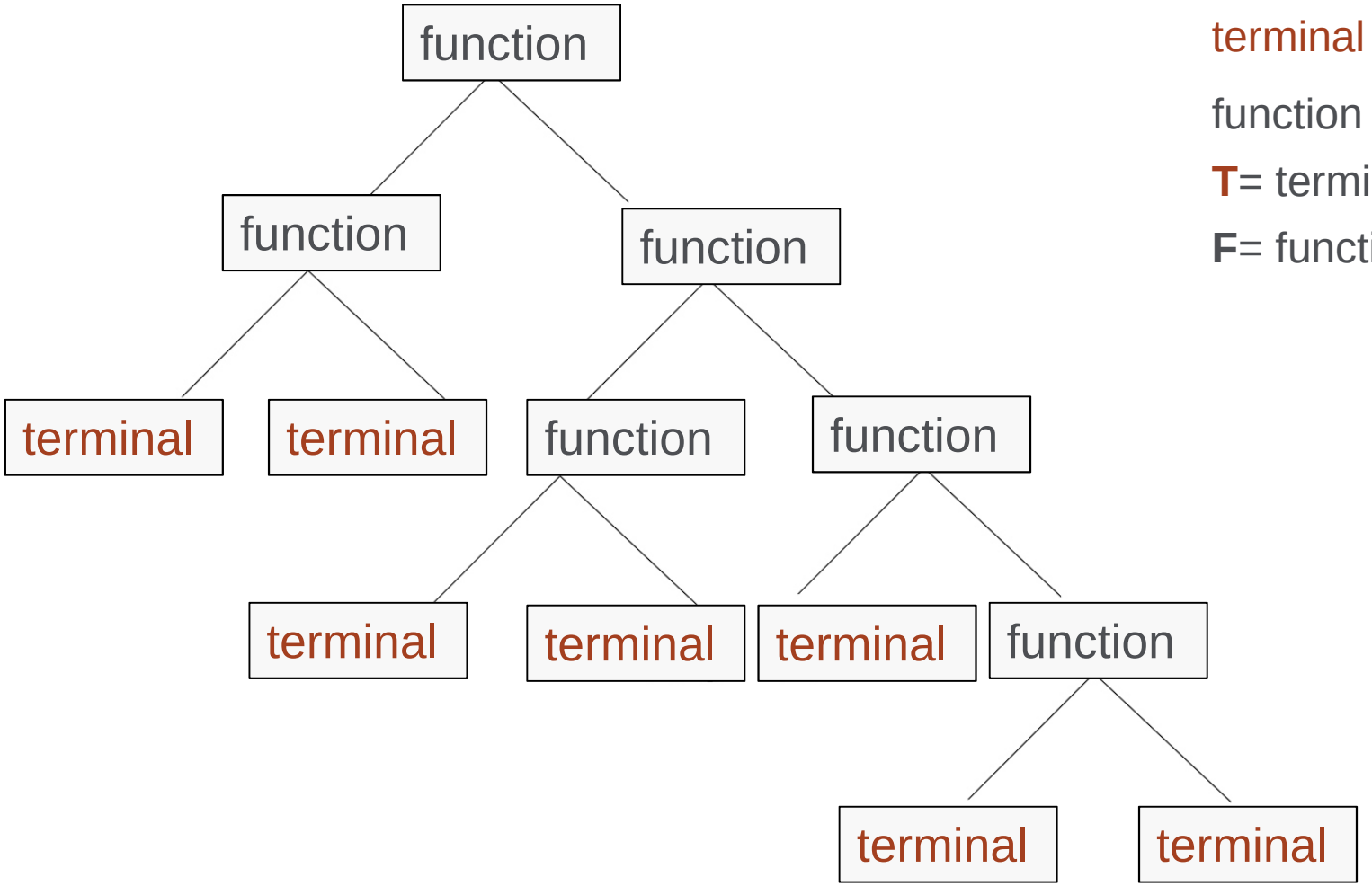
$(x \wedge \text{true}) \rightarrow ((x \vee y) \vee (z \leftrightarrow (x \wedge y)))$

Program




```
i =1;
while (i < 20)
{
        i = i +1;
}
```

The underlying tree structure



terminal ∈ **T**

function ∈ **F**

T= terminal set

F= function set

Trees constitute variable representation

- In GA, ES, EP chromosomes are linear structures (bit strings, integer string, real-valued vectors, permutations)
- Tree shaped chromosomes are non-linear structures
- In GA, ES, EP the size of the chromosomes is **fixed**
- Trees in GP may vary in **depth** and **width**



How are trees built?

- Symbolic expressions can be defined by
  - Terminal set T
  - Function set F

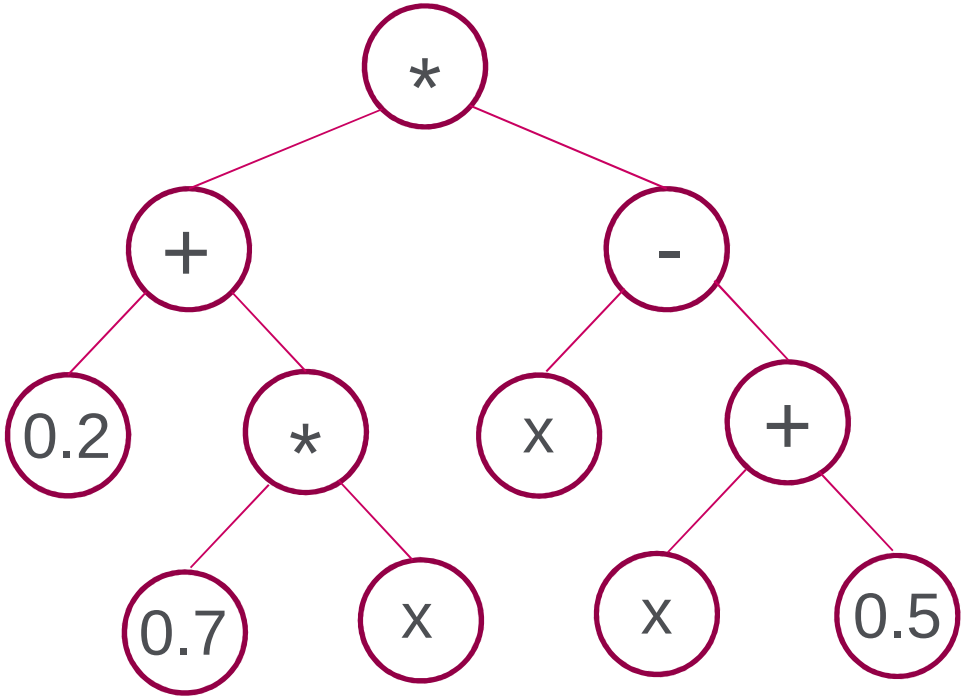
- Adopting the following general recursive definition:
  1. Every $t \in T$ is a correct expression
  2. $f(e_1, \ldots, e_n)$ is a correct expression
     if $f \in F$, f takes n arguments and
     $e_1, \ldots, e_n$ are correct expressions
  3. There are no other forms of correct expressions

- In general, expressions in GP are not typed (closure property: any $f \in F$ can take any $g \in F$ as argument)



Generating an example tree

- T={x, 0,0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1}
- F={ +, -, *}
- Tree depth 3



FITNESS EVALUATION



Fitness evaluation

- There is no evaluation function provided as such, programs have to be **executed** to be evaluated
- If we have a set of desired (input, output) pairs, how do we measure the quality of a program?

- For **fitness case** (x_i, y_i), the program produces $gp(x_i)$
- The difference between the result and the desired output is $\left| gp(x_i) - y_i \right|$

- A potential error measure

$$\sqrt{\sum_{i=1}^{n} (gp(x_i) - y_i)^2}$$

to be minimised

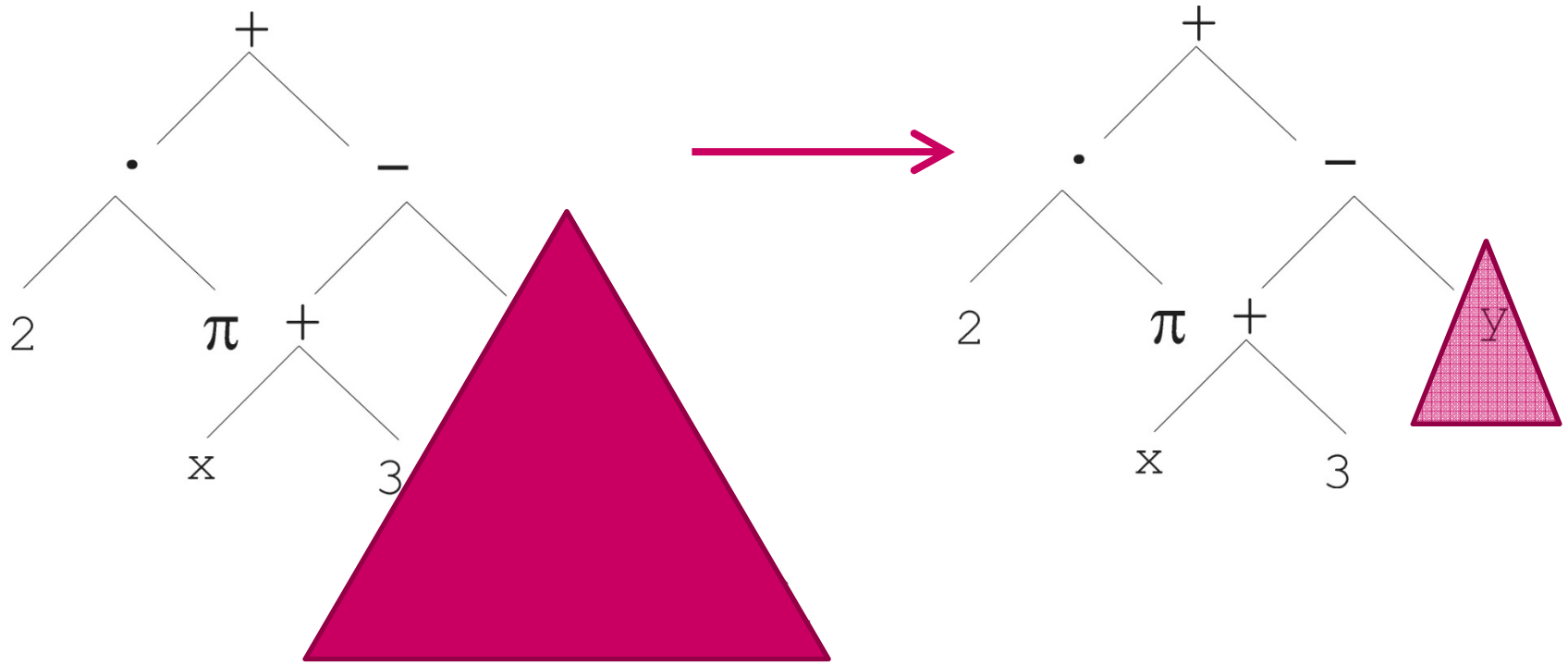


VARIATION OPERATORS



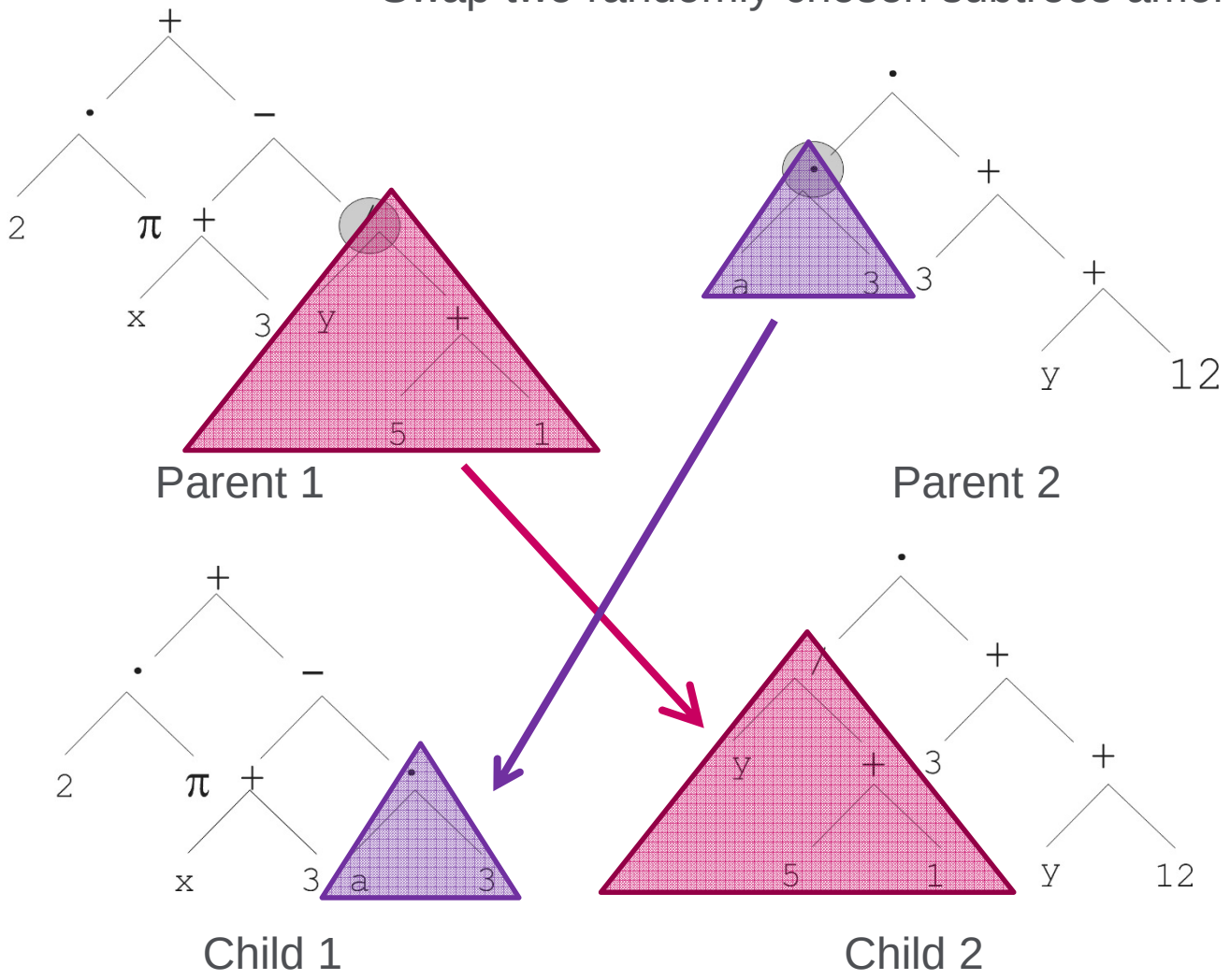
Tree mutation

- **Subtree mutation:** replace randomly chosen subtree by randomly generated tree



Tree recombination

Swap two randomly chosen subtrees among the parents



Interesting points about tree operators

- Mutation has two parameters:
  - Probability p_m to choose mutation
  - Probability to chose an internal node as the root of the subtree to be replaced

- p_m is advised to be very small, 0.05 (Banzhaf et al. ’98)

- Recombination has two parameters:
  - Probability p_c to choose recombination
  - Probability to chose an internal node within each parent as crossover point

- The size of offspring can exceed that of the parents



HOW DOES GP WORK IN PRACTICE?

Step by step symbolic regression

- Given a set of pairs of numbers (x_i, y_i), find the function $f(x)$ so that $f(x_i) = y_i$

- Example

  (-1,1) (-0.9, 0.91) (-0.8, 0.844) (-0.7, 0.79) … (-0.1, 0.91)
  (0,1) (0.1, 1.11)…(0.8, 2.44) (0.9, 2.71) (1, 3)

Let's use functions: +,-,*, % (protected division)

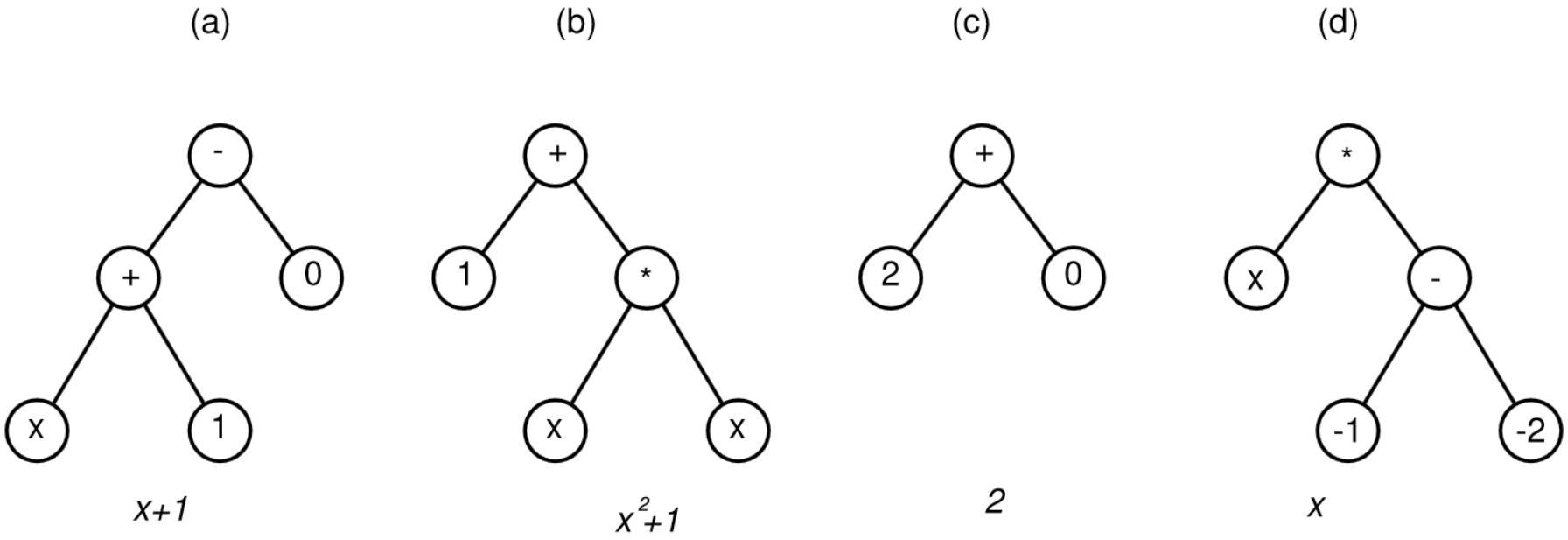
terminals: x and constants from the range [-5, 5]

Population size: 4

Crossover to generate 2 offspring, mutation one, reproduction one



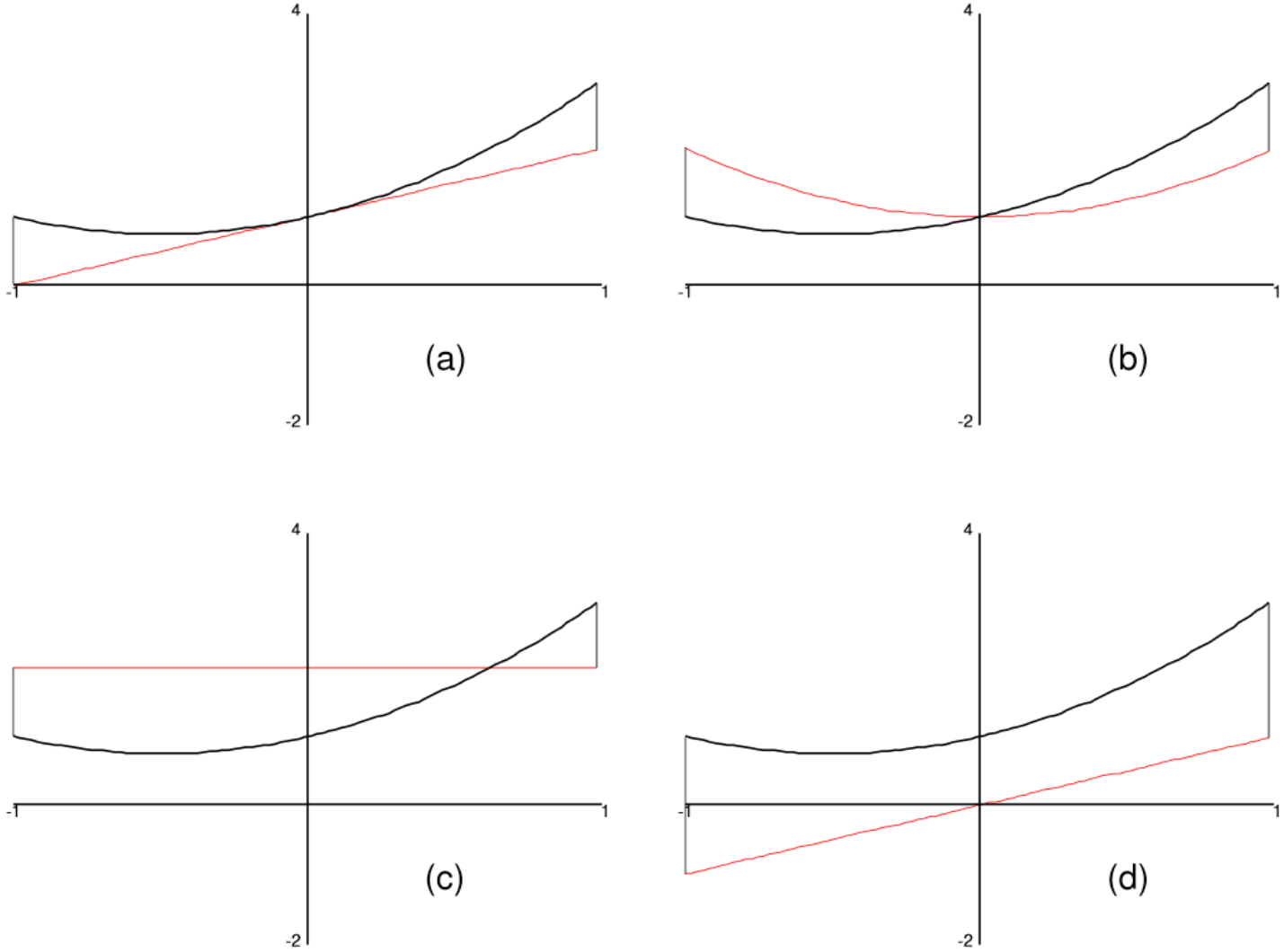
Initial population



Fitness values

7.7	11	17.98	28.7

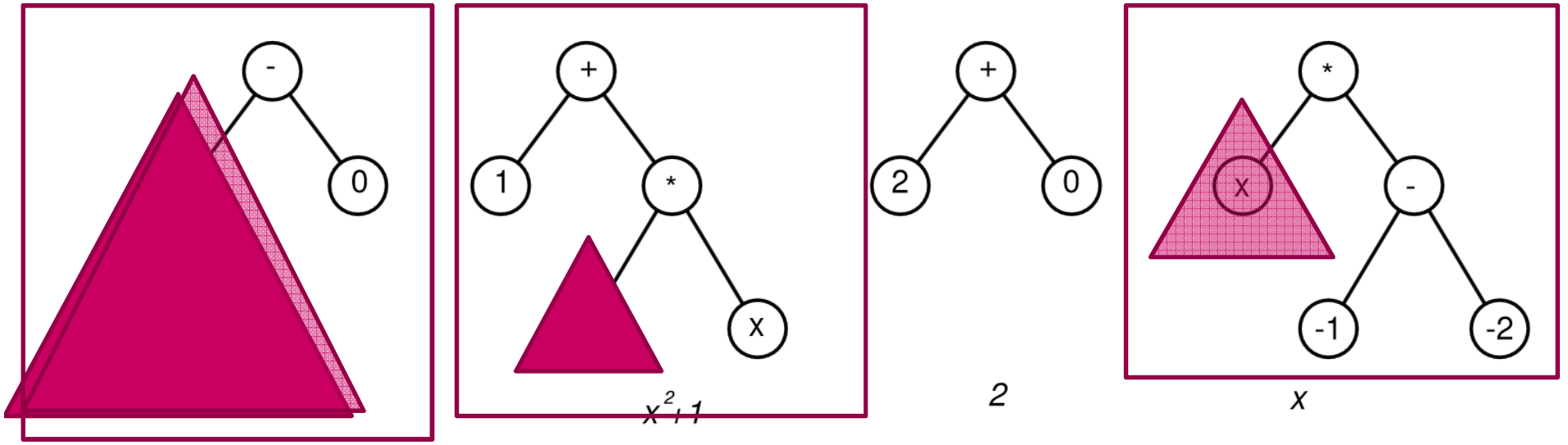
The generation 0 functions



Producing generation 1

Gets copied

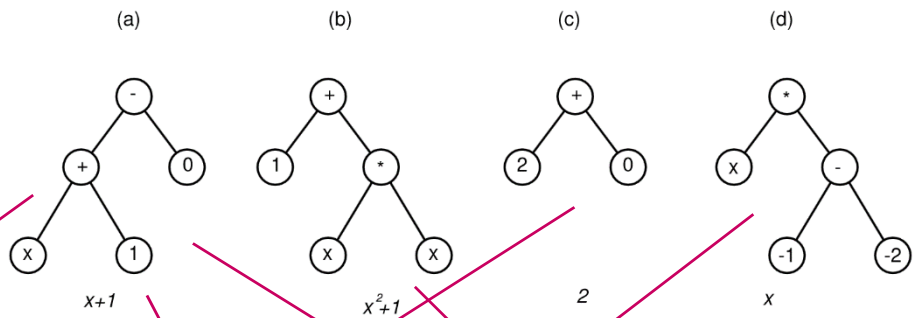
Gets mutated



Fitness values

7.7 11 17.98 28.7

Generation 1

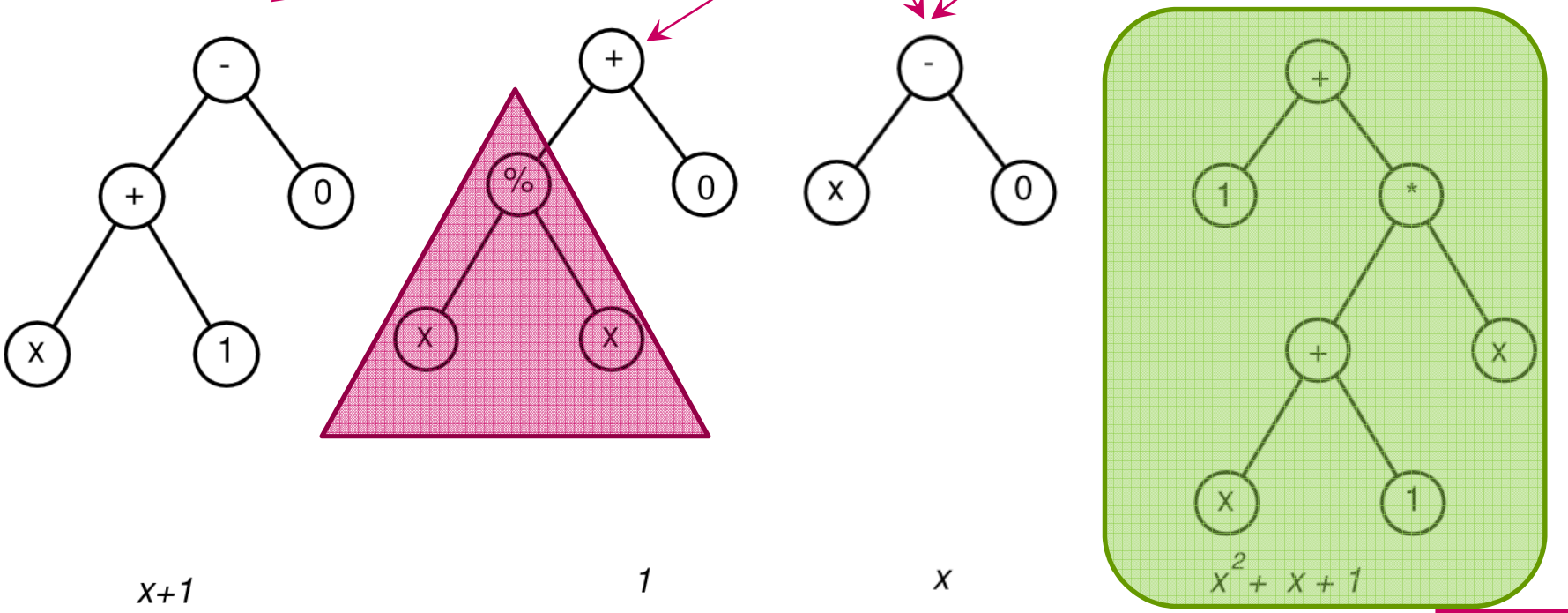


(a) copied from generation 0 (a)

(c) from gen 0 mutated (b)

(a) and (d) crossed over (c)

(b) and (a) crossed over (d)



Perfect solution!

TRICKS OF THE TRADE



(Some) Tricks of the trade

- Presenting results should not only consist of quality but also include the actual (cleaned up) computer programs
- Genetic programs have a tendency to grow, if left uncontrolled (due to selection of leaf nodes for mutation or crossover)
- Simple mechanisms to limit code growth should be employed (for example not allowing offspring to be substantially larger than parents)
- Diversity should be maintained, possibly adaptively



When can we expect GP to do well?

- The relationships between variables are not known or poorly understood
- Finding the size and shape of the solution is an essential part of the problem
- There are good simulators that test the quality of a solution but there are no methods to directly obtain solutions
- There is no conventional mathematical solution available
- An approximate solution is acceptable



THE HUMIES

The Humies: human-competitive results

One of 8 criteria must be satisfied, some of which are:

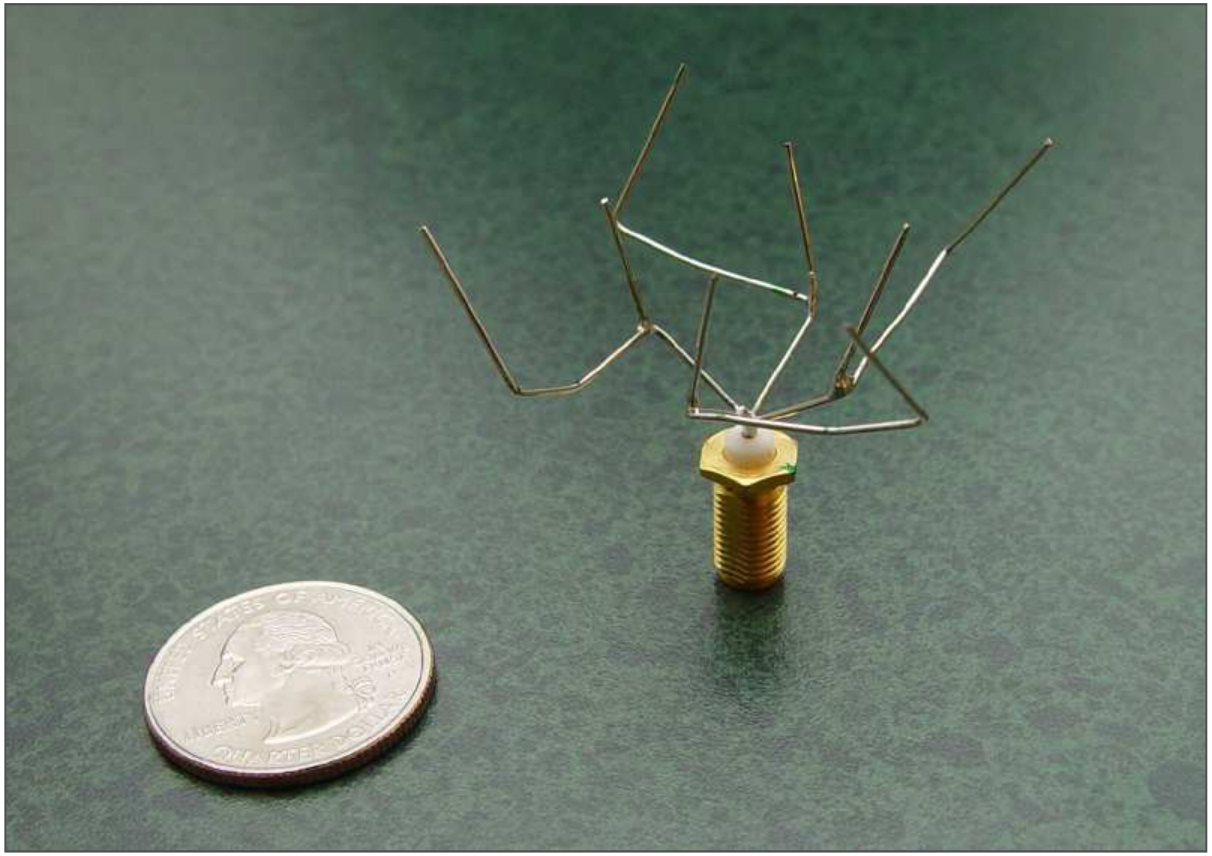
- The result was patented as an invention in the past, is an improvement over a patented invention, or would qualify today as a patentable new invention
- The result is publishable in its own right as a new scientific result — independent of the fact that the result was mechanically created
- The result is equal to or better than a result that was considered an achievement in its field at the time it was first discovered
- The result solves a problem of indisputable difficulty in its field

- Probably ~100 examples *(we at Aston have one)*



Gold medal winner in first year (2004)

Antenna design for Deployment on NASA's Space Technology 5 Mission



Summary

- GP takes inspiration from Natural evolution and tries to achieve automatic programming
- Representation is variable length
- Fitness evaluation and variation operators are specifically defined



Summary cont’d

- GP is good at solving problems where the structure and size of the solution is an important part of the problem
- There are many encouraging human-competitive results
- Systematically designed experiments with statistically validated results are the norm



Recommended reading

- A. E. Eiben, J. E. Smith: Introduction to Evolutionary Computing, GP related parts of Chapters 4, 6

- The book's website
  www.evolutionarycomputation.org

- Poli, Langdon, McPhee & Koza (2007): Genetic Programming: An Introductory Tutorial and a Survey of Techniques and Applications

https://www.researchgate.net/publication/226040746_Genetic_Programming_An_Introduction_and_Tutorial_with_a_Survey_of_Techniques_and_Applications

- Background:
  Humies: http://www.human-competitive.org/

  Genetic programming bibliography (over 14000 entries)
  http://www.cs.bham.ac.uk/~wbl/biblio/

